

# ESPECIFICAÇÃO DA LINGUAGEM JSS (JAVASCRIPT SIMPLIFICADO)

Autores: João Vinicius de Sousa Cabral e Caio Victor.

Este documento define formalmente a especificação da linguagem **JSS (JavaScript Simplificado)**, cobrindo seus aspectos léxicos, gramática sintática (sintaxe) e regras semânticas de validação.

## 1. ESPECIFICAÇÃO LÉXICA (LEXER)

O analisador léxico da linguagem JSS converte o texto do programa em um fluxo de tokens.

### 1.1 Palavras Reservadas (Keywords)

A linguagem possui distinção entre maiúsculas e minúsculas (*case-sensitive*) para as seguintes palavras-chave:

- **Declaração de variáveis/constantes:** let, const
- **Funções e escopo:** function, return
- **Estruturas condicionais:** if, else
- **Estruturas de repetição:** while, for, break
- **Orientação a objetos:** class, constructor, new, this
- **Literais e tipos:** null, true, false, int, real, str, bool, void
- **Entrada de dados:** input

### 1.2 Tokens e Símbolos

- **Operadores Aritméticos:** +, -, \*, /, %, (exponenciação).
- **Operadores Relacionais:** ==, !=, >, >=, <, <=.
- **Operadores Lógicos:** &&, ||, ! (negação).
- **Operadores de Atribuição:** =, +=, -=, \*=, /=, %=, &&=, ||=.
- **Incremento/Decremento:** ++, -- (suportados apenas de forma pré-fixada, ex: ++x).
- **Delimitadores:** (, ), {, }, [, ], ;, ,, ..

### 1.3 Literais

- **Inteiro (INT\_LITERAL):** Uma sequência de dígitos (ex: 123, 0).

- **Real (REAL\_LITERAL):** Números com ponto decimal e opcionalmente notação científica (ex: 1.5, -10.8E2).
- **String (STRING\_LITERAL):** Caracteres delimitados por aspas duplas. Suporta as seguintes seqüências de escape: \n, \", \\. Qualquer outro escape resulta em erro léxico.
- **Identificadores (ID):** Devem iniciar com letra ou sublinhado (\_), seguidos por letras, dígitos ou sublinhado. Identificadores inválidos que iniciam com dígitos (ex: 1a) ou caracteres especiais não suportados (ex: @) geram erros léxicos.

## 1.4 Comentários e Espaçamentos

- Apenas comentários de linha única iniciados por // são permitidos. Comentários multilinha (/\* ... \*/) geram erro léxico explícito instruindo o uso de comentários de linha.
- Espaços em branco e tabulações são ignorados.

## 2. GRAMÁTICA SINTÁTICA (PARSER)

Abaixo está a gramática sintática da JSS escrita na notação EBNF.

EBNF

```

Program          ::= StatementList
StatementList    ::= Statement ( Statement )*
Statement        ::= VarDeclaration
                  | IfStatement
                  | WhileStatement
                  | ForStatement
                  | BreakStatement
                  | Block
                  | FunctionDeclaration
                  | ReturnStatement
                  | ClassDeclaration
                  | ConsoleLogStatement
                  | InputStatement
                  | ExpressionStatement

Type             ::= "int" | "real" | "str" | "bool" | ID
ReturnType       ::= Type | "void"
IdList           ::= ID ( "," ID )*
DimensionList     ::= "[" INT_LITERAL "]" ( "[" INT_LITERAL "]" )*
```

```

VarDeclaration      ::= "let" Type ID ( "=" Expression )? ";"
                    | "const" Type ID "=" Expression ";"
                    | "let" Type DimensionList ID ( "=" Expression
)? ";"
                    | "const" Type DimensionList ID "=" Expression
";"
                    | "let" Type IdList ";"

IfStatement         ::= "if" "(" Expression ")" Block ( "else" (
Block | IfStatement ) )?
WhileStatement      ::= "while" "(" Expression ")" Block
ForInit             ::= VarDeclarationNoSemicolon | Expression |
empty
ForStatement        ::= "for" "(" ForInit ";" ( Expression )? ";" (
Expression )? ")" Block

Block               ::= "{" StatementList? "}"

FunctionDeclaration ::= "function" ReturnType ID "(" ParamList ")"
Block
                    | "function" Type DimensionList ID "("
ParamList ")" Block

ParamList           ::= ( Param ( "," Param )* )?
Param               ::= Type ID
                    | Type DimensionList ID

ReturnStatement     ::= "return" ( Expression )? ";"

ClassDeclaration    ::= "class" ID "{" ClassMemberList "}"
ClassMemberList     ::= ClassMember*
ClassMember         ::= ClassAttribute | ClassConstructor |
ClassMethod

ClassAttribute      ::= Type ID ";"
                    | Type DimensionList ID ";"

ClassConstructor    ::= ID "constructor" "(" ParamList ")" Block

ClassMethod         ::= Type ID "(" ParamList ")" Block
                    | "void" ID "(" ParamList ")" Block
                    | Type DimensionList ID "(" ParamList ")"

```

Block

ConsoleLogStatement ::= "console.log" "(" ArgumentList ")" ";"

InputStatement ::= "input" "(" ArgumentList ")" ";"

ExpressionStatement ::= Expression ";"

ArgumentList ::= ( Expression ( "," Expression )\* )?

Expression ::= Expression BinaryOp Expression  
| UnaryOp Expression  
| Expression "[" Expression "]"  
| Expression "." ID  
| "this" "." ID  
| ID  
| INT\_LITERAL  
| REAL\_LITERAL  
| STRING\_LITERAL  
| "true"  
| "false"  
| "null"  
| "(" Expression ")"  
| "new" ID "(" ArgumentList ")"  
| Expression "(" ArgumentList ")"  
| "[" ArgumentList "]"  
| Type "(" Expression ")"

### 3. REGRAS SEMÂNTICAS (SEMANTIC)

O analisador semântico realiza checagens de tipos, escopos e atribuições.

#### 3.1 Regras de Escopo

- **Escopo Global:** Funções, classes e variáveis declaradas no nível raiz são globais. Não é permitida a declaração de funções ou classes aninhadas.
- **Escopo de Bloco:** Variáveis e constantes declaradas com `let` ou `const` dentro de chaves { ... } são locais e visíveis apenas no bloco e em sub-blocos internos.
- **Re-declaração:** É estritamente proibido redeclarar um identificador no mesmo escopo local.

## 3.2 Sistema de Tipos e Coerção

- **Tipos válidos:** `int`, `real`, `str`, `bool`, tipos de classes declaradas e arranjos (`tipo[]`).
- **Coerção Implícita:**
  - `int` é implicitamente convertido para `real` quando usado com um operando `real` em expressões aritméticas.
  - Qualquer tipo primitivo (`int`, `real`, `bool`, `str`) é implicitamente convertido para `str` quando operado com `+` de concatenação onde um dos operandos é `str`.
- **Casting Explícito:** Suporta conversões explícitas usando as funções globais `int()`, `real()`, `bool()` e `str()`.

## 3.3 Funções e Métodos

- A função `main` (quando declarada globalmente) é opcional. Se presente, deve ter a assinatura estrita `function void main()` (sem parâmetros e com retorno `void`).
- Funções não-void devem possuir obrigatoriamente um comando `return` com expressão compatível em seu corpo.
- O tipo de retorno de funções ou métodos **não pode ser um vetor/arranjo**.

## 3.4 Classes e Objetos

- Toda classe declarada deve obrigatoriamente definir um método construtor (`constructor`).
- **Ordem de Declaração:** Os atributos da classe devem vir antes do construtor, e o construtor deve vir antes dos métodos.
- O nome do construtor deve coincidir exatamente com o nome da classe.
- Objetos de classe iniciam com valor padrão `null` e podem ser comparados logicamente com `null`.

## 3.5 Segurança e Validação de Vetores

- **Declaração:** Vetores devem ser declarados com tamanho constante estático definido em tempo de compilação (ex: `let int[5] v;`).
- **Atribuição Direta:** Não é permitida atribuição direta entre estruturas de vetores (ex: `v = outro_vetor;`). Vetores devem ser modificados ou copiados elemento a elemento.
- **Compatibilidade de Parâmetros:** Ao passar um vetor como parâmetro para uma função ou construtor de classe, o compilador valida se o tamanho e dimensão do

vetor passado condizem exatamente com o esperado pela assinatura da função (mecanismo para evitar *buffer overflow*).

- **Validação Estática de Limites (Out of Bounds):** Acessos a elementos de vetores com índices inteiros constantes (literais inteiros) são validados estaticamente em tempo de compilação. Se o índice for menor que zero ou maior/igual ao tamanho declarado, o compilador gera um erro semântico impeditivo.